



AULA 1

**Introdução à engenharia de
software e gestão ágil de projetos**



O PROFESSOR

Marcos Kalinowski

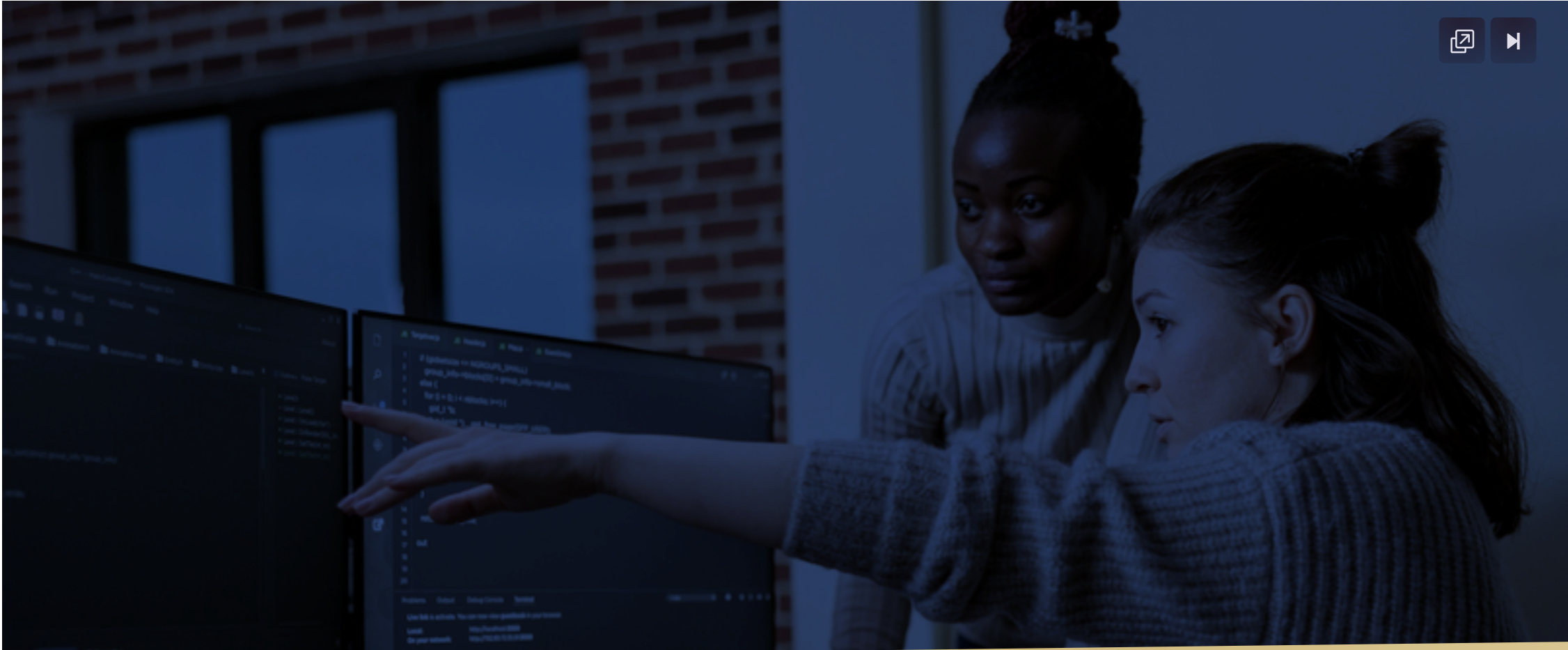
Professor do quadro principal do Departamento de Informática da PUC-Rio, onde orienta pesquisas de mestrado, doutorado e pós-doutorado e coordena projetos de pesquisa e desenvolvimento junto a empresas como Americanas S.A. e Petrobras nas áreas de engenharia de *software* e ciência de dados. Doutor em engenharia de sistemas e computação pela COPPE/UFRJ. Atuou por mais de 10 anos na indústria de *software* antes de se tornar professor (como desenvolvedor, consultor e diretor). Forneceu treinamentos *in-company* para empresas como Chemtech, CNEN, DataPrev, Loggi, Oceaneering, ONS, Petrobras, Sakonnet Technology, entre outras. É *senior advisor* da equipe técnica responsável pelo programa nacional MPS.BR. Avaliador líder do modelo MPS-SW.



CONVIDADO

William Ducca Fernandes

Senior Data Scientist na Microsoft. Antes disso atuou como líder de pesquisa e desenvolvimento na iniciativa de transformação digital ExACTa PUC-Rio, onde liderou projetos junto a diversos parceiros da indústria, com destaque para sua participação em dezenas de entregas para a Petrobras e a geração de produtos inovadores com patentes depositadas. Tem doutorado em Informática na área de Ciência de Dados pelo Departamento de Informática da PUC-Rio e mais de 10 anos de experiência prática atuando em projetos de ciência de dados e *machine learning*.

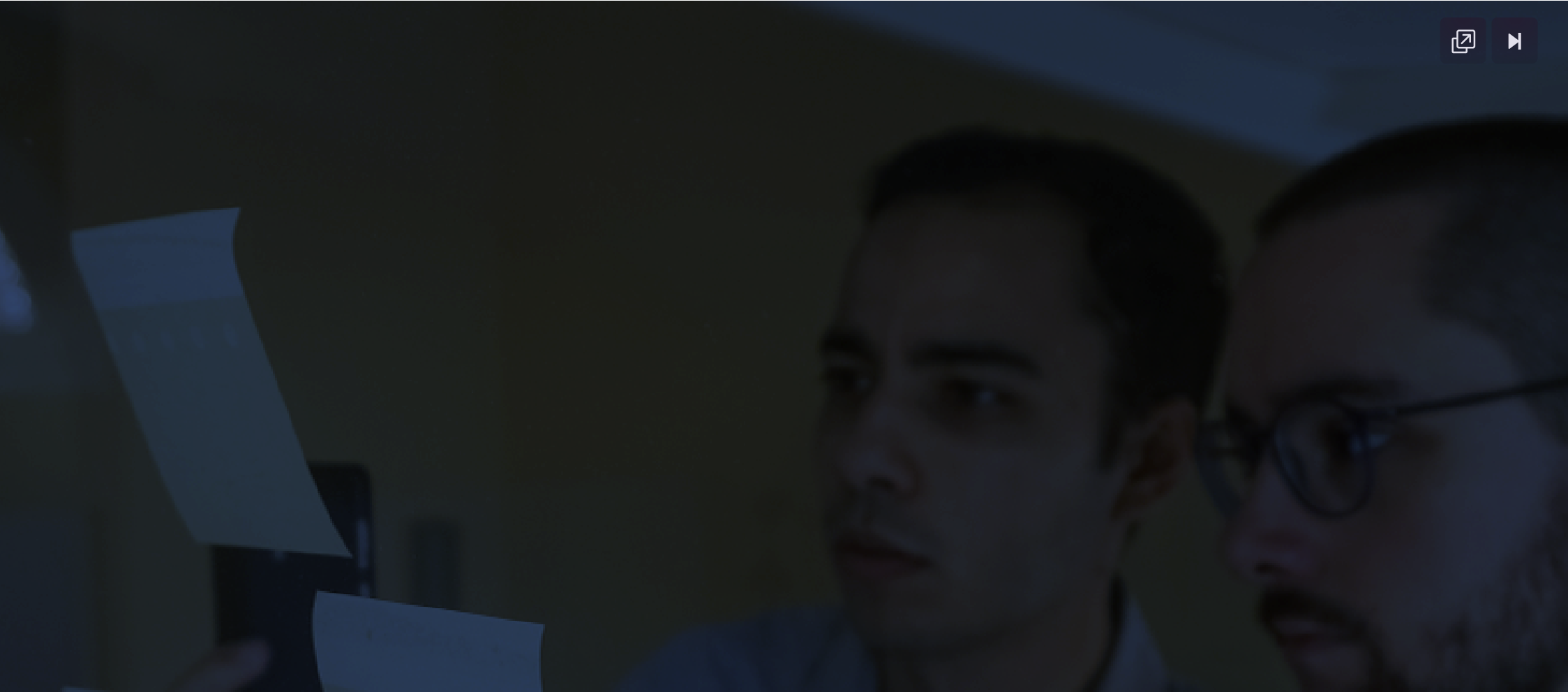


Objetivos de aprendizagem da aula

Ao final desta aula, você irá:

- Compreender o que é engenharia de *software* e por que ela é particularmente importante no contexto de ciência de dados.
- Entender o funcionamento geral da gestão ágil de projetos de *software*.

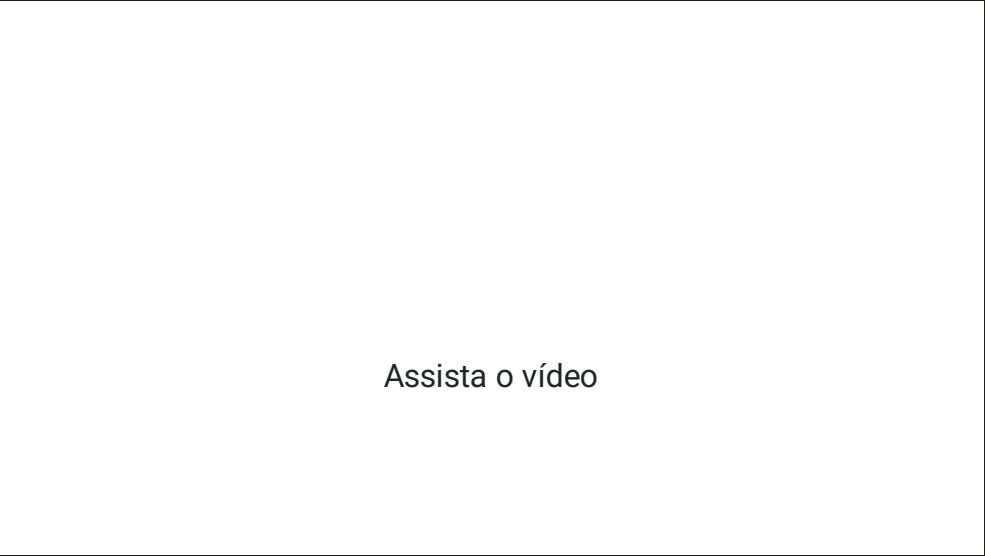
Nesta primeira aula, teremos o primeiro contato com o conceito de engenharia de *software* e já aprenderemos algumas práticas de mercado. Contextualizaremos o cenário atual e discutiremos por que a engenharia de *software* é particularmente importante no contexto de ciência de dados. Veremos como a engenharia de *software* se faz presente nos principais modelos e normas de referência para o desenvolvimento de *software*. De posse dessa compreensão, enfatizaremos abordagens modernas e ágeis ao longo da disciplina. Ainda nesta aula, iremos compreender o funcionamento geral da gestão ágil de projetos de *software*, amplamente adotada pelo mercado.



Que história é essa?

No vídeo a seguir, você verá um pouco sobre o uso de métodos ágeis para desenvolver sistemas de *software* inteligentes, além de saber mais sobre a importância de boas práticas da engenharia de *software* para o desenvolvimento de sistemas de *software* inteligentes. Vamos lá?

React de Mercado



Assista o vídeo

Introdução à engenharia de *software*

O que é engenharia de *software*?

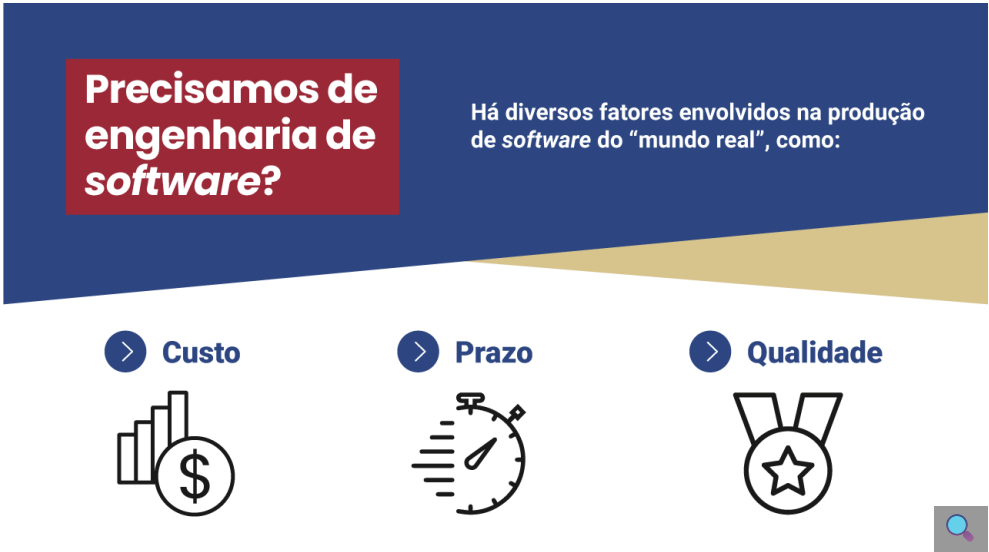
Para aqueles que vêm de uma área de conhecimento diferente da computação, a primeira coisa que devem estar se questionando é: o que é essa tal de engenharia de *software*?



Mantendo o foco!

De forma resumida, engenharia de *software* é a aplicação de princípios usados no campo da engenharia, que geralmente lida com sistemas físicos, para especificação, projeto, desenvolvimento, teste, implantação e gerenciamento de sistemas de *software*.

Considerando a construção de sistemas de *software*, a programação é uma parte importante do processo, mas não é tudo. Precisamos também investir esforços em entender o que programar (requisitos), como programar (arquitetura e projeto), se o que foi programado está certo (revisões e teste de *software*) e como implantar a solução de forma eficiente em operação (DevOps).



Em virtude do tamanho do *software*, os fatores **custo, prazo e qualidade** podem ser difíceis de garantir. É muito mais fácil, por exemplo, estimar o esforço para o desenvolvimento de uma pequena agenda pessoal do que para um sistema de gestão integrada envolvendo dados de várias áreas de negócio.

De fato, há diversas evidências científicas apontando para benefícios da aplicação de boas práticas da engenharia de *software* no custo, no prazo e na qualidade de projetos de *software*. Em uma investigação realizada ao longo de oito anos junto a centenas de empresas desenvolvedoras de *software* (TRAVASSOS; KALINOWSKI, 2014), observamos benefícios de aumento da produtividade e de redução de retrabalho decorrentes de investimentos em princípios de engenharia para desenvolver um *software* correto por construção. Ademais, os esforços de manutenção tendem a ser significativamente reduzidos quando um *software* foi construído com os devidos cuidados de arquitetura e projeto da solução.

Estima-se que, em projetos de *software*, o esforço de retrabalho esteja entre 30 e 40% do esforço total do projeto. Isso seria inimaginável para projetos em outras áreas, como, por exemplo, na engenharia civil. Assim, a aplicação de boas práticas da engenharia de *software* costuma dar resultados imediatos. Por exemplo, recentemente aumentamos a produtividade de uma das nossas empresas parceiras

ao reduzirmos o retrabalho para um terço do que era antes da nossa intervenção. Basicamente o que fizemos foi trazer boas práticas para a especificação dos sistemas e o seu controle da qualidade.



Mas precisamos de engenharia de *software* para ciência de dados?

Sabemos que hoje soluções baseadas em *software* permeiam as nossas vidas em quase todos os aspectos. Além disso, os avanços contemporâneos em *machine learning* (ML) e a disponibilidade de grandes quantidades de dados têm tornado viável e de relevância prática incorporar componentes de ML em sistemas de *software*. Esses componentes permitem às soluções de *software* aprender a partir dos dados e realizar classificações, previsões, agrupamentos e associações inteligentes. Daqui em diante, chamaremos esses sistemas de **sistemas de *software* inteligentes**.

Anos atrás, havia um questionamento se fazer *software* não era mais arte do que engenharia, considerando que a criatividade do programador é uma parte importante do processo. Este mito foi desconstruído para sistemas convencionais com base em evidências diversas reforçando a importância e os benefícios da engenharia de *software*. Hoje, é comum se escutar o mesmo argumento para sistemas de *software* inteligentes, em particular para a construção dos componentes de ML, que muitas vezes envolvem modelos construídos por profissionais de outras áreas que não a computação em si, como biólogos, economistas, estatísticos, entre outros.

Ao comparar *software* com arte, precisamos pensar nas consequências de um *software* conter defeitos que resultem em falhas quando submetido à operação. Em geral, soluções são embutidas em sistemas de *software* para permitir sua sistematização e aplicação em larga escala. Dessa forma, *softwares* que apresentam falhas em operação podem literalmente sistematizar o caos.

Embora muitas vezes isso não seja claro para a sociedade como um todo, as consequências de falhas podem ser ainda mais drásticas no caso de sistemas de *software* inteligentes.



Importância da engenharia de software para a ciência de dados

Vejamos, no vídeo a seguir, alguns casos de falhas em sistemas de software inteligentes e suas consequências, para exemplificar a importância da engenharia de software também nesse contexto.

Processo de engenharia de software

Em geral, uma das metas da engenharia de *software* é definir uma abordagem ou um processo. Isto é, definir a maneira como o *software* será construído, incluindo as boas práticas a serem seguidas durante a construção. **Um processo de engenharia de *software* tem como objetivo garantir a produção de *software* de alta qualidade que está de acordo com as necessidades dos seus usuários finais com um cronograma e custo previsível.**



Normalmente, esse processo é composto por um conjunto de atividades seguindo um modelo de ciclo de vida, bem definidas, com dependências entre si e ordem de execução, com responsáveis e artefatos de entrada e saída.



É desejável, ainda, que as atividades tenham uma descrição sistemática das tarefas a serem realizadas.

Clique nas abas para conhecer os modelos de ciclo de vida



Interativo

Descrição do interativo

O **modelo cascata** assume a execução sequencial das atividades e, em geral, se mostra adequado apenas para projetos pequenos, por exemplo, envolvendo menos de 200 horas. Isto porque ele presume que é possível realizar toda a especificação de uma só vez, validar essa especificação e, então, seguir para a atividade seguinte. Na prática, essa natureza sequencial raramente consegue ser seguida em projetos de médio ou grande porte. O modelo incremental e o evolutivo apresentam alternativas.

No **modelo incremental**, os requisitos são segmentados em uma série de incrementos. A cada incremento é produzida uma versão operacional do *software*. O processo se repete até que um produto completo seja produzido. A segmentação de requisitos é geralmente planejada *a priori*. Dessa forma, menor custo e menos tempo são necessários para entregar a primeira versão, e os riscos associados ao desenvolvimento são menores, devido ao tamanho reduzido de cada incremento. O número de solicitações de mudança nos requisitos também pode diminuir, devido ao curto tempo de desenvolvimento dos incrementos. Este é o modelo de ciclo de vida utilizado nas variantes do **processo unificado**.

Aprenda mais

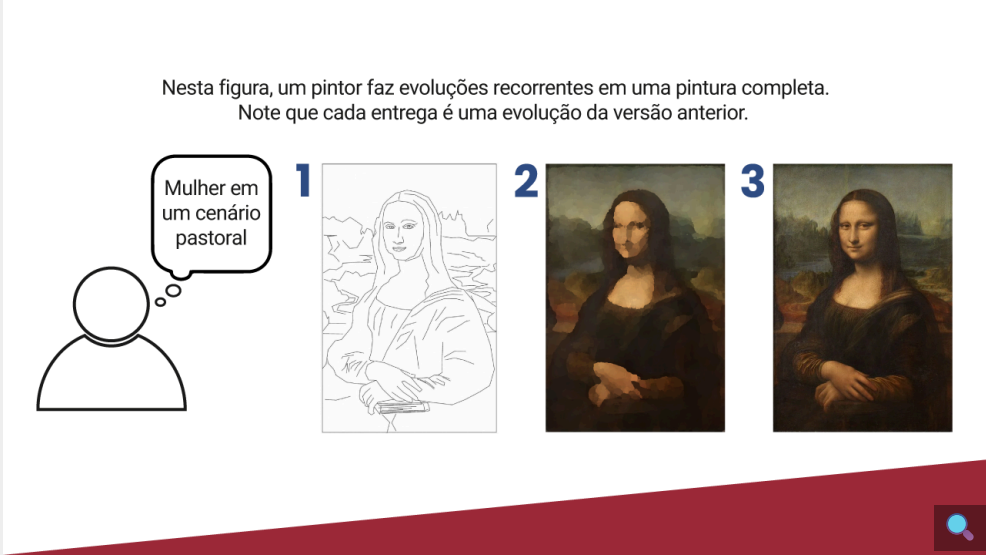
Processo unificado é um nome genérico para uma família de modelos de processo orientados a plano (as iterações são planejadas *a priori*) iterativos e incrementais. O mais conhecido entre esses modelos de processo é o **rational unified process (RUP)**, um refinamento do processo unificado criado pela *Rational Software* (atualmente da IBM).

A figura a seguir ilustra a dinâmica do modelo incremental.



No **modelo evolutivo**, são desenvolvidas versões parciais que atendem aos requisitos conhecidos inicialmente. A primeira versão é, então, usada para refinar os requisitos para uma segunda versão, e assim por diante. A partir do conhecimento sobre os requisitos, obtido com o uso, continua-se o desenvolvimento, evoluindo o **produto**. Esse ciclo de vida tende a se mostrar adequado quando os requisitos não podem ser completamente especificados de início e o uso do sistema pode aumentar o conhecimento sobre o produto e melhorar os requisitos. Mas é preciso ter cuidado na sua aplicação. Além de poder gerar muito retrabalho, os usuários podem não entender a natureza da abordagem e se decepcionar quando os resultados não são satisfatórios.

A figura a seguir ilustra a dinâmica do modelo evolutivo.

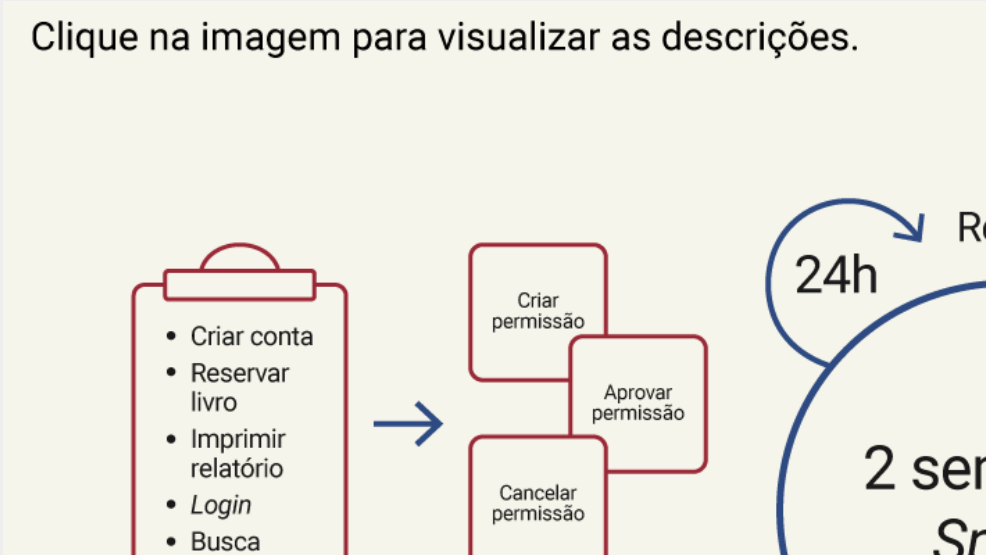


O **ciclo de vida associado ao Scrum** é hoje o mais amplamente **utilizado**. Por essa razão, tendo em vista a formação objetiva e focada no mercado, ele será o mais detalhado nesta disciplina. De fato, teremos uma seção inteira sobre a gestão ágil de projetos de *software* ao final desta aula. Por enquanto, basta compreender a natureza desse ciclo de vida. O Scrum é um *framework* de gestão ágil iterativo e incremental/evolutivo muito aplicado no contexto de desenvolvimento de *software*.

A dinâmica do Scrum está ilustrada na imagem a seguir.



Clique na imagem para visualizar as descrições.



Interativo

Descrição do interativo

Ao final de um *sprint*, ocorre a **revisão do *sprint*** e a **retrospectiva do *sprint***, na qual todos os membros da equipe revisam a entrega e refletem sobre o *sprint* passado. O Scrum é facilitado por um **Scrum master**, que tem como função primária remover qualquer impedimento à habilidade de uma equipe de entregar o objetivo do *sprint*.

Neste ponto em que já compreendemos as dinâmicas típicas de processos e abordagens de desenvolvimento de *software*, veremos que tipo de práticas mais específicas a engenharia de *software* contempla.

Práticas de engenharia de software

Existem diversos modelos de referência, normas e corpos de conhecimento que organizam práticas de engenharia de *software*.

Mantendo o foco!

Entre os modelos de referência, destacamos o modelo de referência brasileiro, que é o **modelo MPS para *software* (MPS-SW)**, do programa para a Melhoria do Processo de Software Brasileiro (MPS.BR), e o **Capability Maturity Model Integration (CMMI)**, da *Information Systems Audit and Control Association* (ISACA), principal modelo de referência internacional. Entre as normas, a **ISO 12207 – Systems and Software Engineering – Software Life Cycle Processes** representa a norma ISO mais significativa para a engenharia de *software*. Além disso, existe um corpo de conhecimento organizado pela IEEE Computer Society, o **Software Engineering Body of Knowledge (SWEBOK)**.



Como existe certa correspondência e compatibilidade entre as práticas propostas por esses modelos, normas e corpos de conhecimento, ilustraremos as práticas de engenharia de *software* com base no MPS-SW. **É importante ressaltar que existe evidência científica de que os resultados da adoção de boas práticas contidas nesses tipos de modelo de referência tendem a ser positivos na prática (KALINOWSKI *et al.*, 2015).**

Essencialmente, o MPS-SW contempla boas práticas da engenharia de *software*, organizadas em processos. A figura a seguir, extraída do Guia Geral de MPS para *Software* de 2021 (SOFTEX, 2021), apresenta os **processos de projeto** e os **processos organizacionais** desse modelo. Esses processos fornecem uma boa visão geral de tópicos relevantes para o mercado estudados no contexto da engenharia de *software*.

Processos de projeto	Processos organizacionais
Gerência de projetos	Gerência de recursos humanos
Engenharia de requisitos	Gerência de configuração
Projeto e construção de produto	Gerência organizacional
Integração de produto	Gerência de processos
Verificação e validação	Medição
	Aquisição
	Gerência de decisões

É possível observar que os processos de projeto contemplam processos mais voltados para a gestão e a engenharia do *software*, ao passo que os processos organizacionais podem ser vistos como processos de apoio para o bom funcionamento dos projetos.

Em relação aos processos de projeto, eles organizam práticas de:

- **gerência de projetos** (incluindo práticas de planejamento e estimativa e para acompanhamento dos projetos);
- **engenharia de requisitos** (incluindo práticas para a especificação de *software* e a gestão de mudanças);
- **projeto e construção de produto** (incluindo práticas relacionadas com modelagem, codificação e testes unitários de componentes);
- **integração do produto** (incluindo práticas para determinar a estratégia de integração de componentes e os testes de integração); e
- **verificação e validação** (incluindo práticas de revisão e teste de sistemas e de aceitação).

Ao longo da disciplina, daremos uma visão geral dessas práticas, já iniciando com práticas de gestão ágil de projetos, ainda nesta aula.

Em relação aos processos organizacionais, enfatizaremos aspectos mais práticos relacionados com a **gerência de configuração** (incluindo práticas de controle de versões e de mudanças), por ser um processo fundamental para o bom andamento de projetos de *software*. Informações para os demais processos poderão ser encontradas no modelo, que tem seus guias gratuitamente disponíveis *on-line*.

Um erro comum é confundir os modelos de referência com métodos. Os modelos não limitam os métodos, técnicas ou ferramentas a serem adotados para o desenvolvimento. É possível, por exemplo, implementar as exigências do modelo de referência MPS-SW baseado em métodos orientados a plano, como o RUP, ou ágeis, como o Scrum. O que os modelos contêm é um conjunto de boas práticas, sem delimitar a forma como essas práticas devem ser realizadas.



Gestão ágil de projetos

Manifesto ágil e princípios ágeis

O *Manifesto ágil* é uma declaração de valores e princípios essenciais para o desenvolvimento de *software*. O documento foi criado em fevereiro de 2001, quando 17 profissionais, que já praticavam métodos ágeis, se reuniram no estado norte-americano de Utah.

Embora esses profissionais utilizassem abordagens e métodos diferentes, eles compartilhavam dos mesmos fundamentos e decidiram escrever um documento que serviria como grito de guerra aos novos processos de desenvolvimento de *software*: o **Manifesto ágil** (BECK et al., 2001). Essencialmente, este documento serviu como base para definir o que é e o que não é desenvolvimento ágil de *software*.

Veja, a seguir, os quatro valores fundamentais do *Manifesto ágil*.

Manifesto para desenvolvimento ágil de software

Estamos descobrindo maneiras melhores de desenvolver *software*, fazendo-o nós mesmos e ajudando outros a fazerem o mesmo. Por meio deste trabalho, passamos a valorizar:

- **Indivíduos e interações** mais que processos e ferramentas.
- **Software em funcionamento** mais que documentação abrangente.
- **Colaboração com o cliente** mais que negociação de contratos.
- **Responder a mudanças** mais que seguir um plano.

Ou seja, mesmo havendo valor nos itens da segunda parte das sentenças, valorizamos mais os itens destacados.

Fonte: <https://agilemanifesto.org/iso/ptbr/manifesto.html>

Contendo 12 princípios, o *Manifesto ágil* tornou-se uma espécie de guia que orienta as escolhas de métodos e ferramentas de times ágeis de projetos. Segue uma descrição dos 12 princípios incluídos no *Manifesto ágil*:

1. Nossa maior prioridade é **satisfazer o cliente** por meio da entrega contínua e adiantada de *software* com valor agregado.
2. **Mudanças nos requisitos são bem-vindas**, mesmo que tardiamente no desenvolvimento. Processos ágeis tiram vantagem das mudanças visando à vantagem competitiva para o cliente.



- 3. **Entregar frequentemente *software* funcionando**, de poucas semanas a poucos meses, com preferência à menor escala de tempo.
- 4. **Pessoas de negócio e desenvolvedores devem trabalhar diariamente em conjunto** por todo o projeto.
- 5. Construa projetos em torno de **indivíduos motivados**. Dê a eles o ambiente e o suporte necessários e confie neles para fazer o trabalho.
- 6. O método mais eficiente e eficaz de transmitir informações para e entre uma equipe de desenvolvimento é por meio de **conversa face a face**.
- 7. ***Software* funcionando** é a medida primária de progresso.
- 8. Os processos ágeis promovem **desenvolvimento sustentável**. Os patrocinadores, desenvolvedores e usuários devem ser capazes de manter um ritmo constante indefinidamente.
- 9. Contínua atenção a **excelência técnica** e **bom *design*** aumenta a agilidade.
- 10. **Simplicidade**. Maximizar a quantidade de trabalho não realizado é essencial.
- 11. As melhores arquiteturas, requisitos e designs emergem de equipes **auto-organizáveis**.
- 12. Em intervalos regulares, a equipe **reflete** sobre como se tornar mais eficaz e, então, refina e **ajusta** seu comportamento de acordo.

Cerca de 20 anos depois do *Manifesto ágil*, em 2021, um conjunto de internacional de pesquisadores, incluindo o autor desta aula, revisitou o que significa desenvolvimento ágil em uma pesquisa junto a desenvolvedores de diversos países (KUHRMANN *et al.*, 2022). Nessa pesquisa, foi possível confirmar a predominância contemporânea de desenvolvimento ágil, e concluiu-se que, na percepção dos desenvolvedores, o grau de agilidade depende mais do alinhamento das práticas (gestão de *backlog*, integração contínua, desenvolvimento orientado a testes, entre outras) com os princípios ágeis do que dos métodos em si (Cascata, RUP, Scrum). Entretanto, o método predominante entre equipes ágeis é o Scrum, detalhado a seguir.

Visão geral do Scrum

Como vimos anteriormente, o Scrum é um *framework* de gestão ágil que utiliza tempo fixo (*sprints*) em vez de escopo fixo para determinar seus incrementos. Nesta parte da aula, iremos detalhar mais o Scrum, entendendo seus **pilares**, os **papéis** envolvidos e as **cerimônias**.

Pilares do Scrum

O Scrum se baseia em três pilares: transparência, inspeção e adaptação.



- **Transparência:** aspectos significativos devem estar visíveis aos responsáveis pelos resultados.
- **Inspeção:** os usuários Scrum devem, frequentemente, inspecionar os artefatos Scrum e o progresso para detectar variações.
- **Adaptação:** se um ou mais aspectos desviou do esperado, o processo ou o material que está sendo produzido deve ser ajustado.

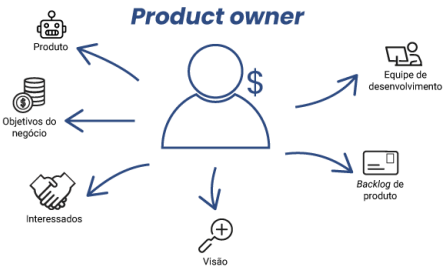
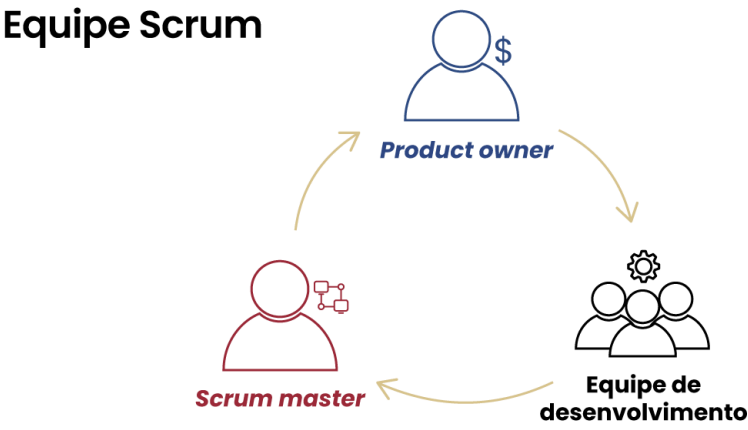
Papéis do Scrum

Os papéis envolvidos no Scrum são o *product owner*, o *Scrum master* e a **equipe de desenvolvimento**.



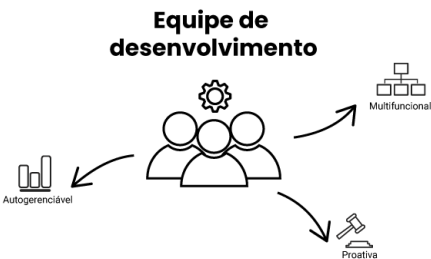
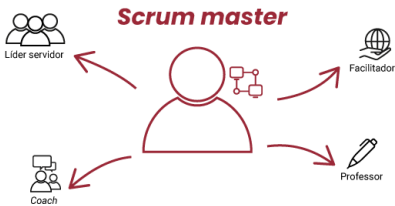
Papéis e responsabilidades

Equipe Scrum



O **product owner (PO)** foca no retorno de investimento (ROI) e no valor agregado para o cliente. É o único que edita e gerencia o *backlog* do produto. A equipe deve respeitar suas decisões, ao passo que suas decisões devem sempre considerar os pontos de vista dos diferentes interessados (*stakeholders*). O PO deve buscar entender bem o produto, os objetivos de negócio e as necessidades dos interessados, ter visão para o produto, atuar próximo à equipe de desenvolvimento e ser o responsável pelo *backlog* do produto.

O **Scrum master**, por sua vez, cuida do processo. Ele aplica o Scrum, facilita os eventos e remove impedimentos do time. É parte do seu papel ensinar a correta aplicação do processo e liderar o time rumo aos objetivos do PO. Conforme ilustrado na figura, o **Scrum master** deve atuar como um líder servidor (que ajuda a realizar, mostra como fazer e remove impedimentos), um *coach* e professor (que orienta e ensina sobre o processo) e como um facilitador das cerimônias.

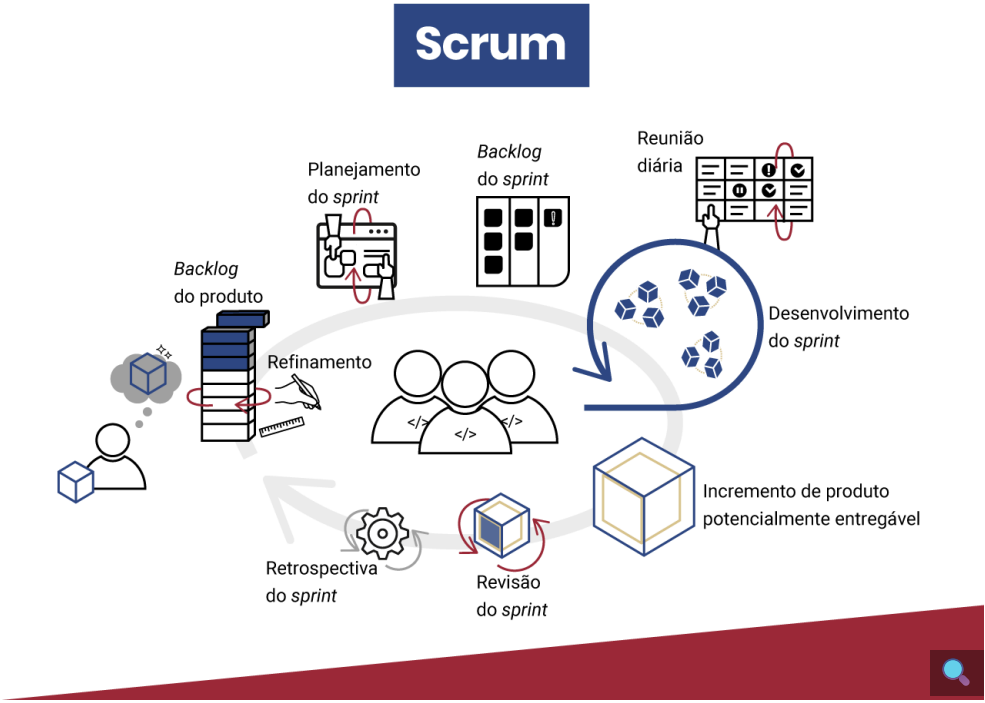


Por fim, a **equipe de desenvolvimento** deve buscar ser proativa e autogerenciável em relação ao atendimento das tarefas do *backlog* do *sprint*. A equipe deve ter atuação multifuncional (dentro do possível de sua *expertise*) e assumir responsabilidade compartilhada pelo desenvolvimento.

Cerimônias do Scrum

Para entender bem a dinâmica do Scrum, é fundamental entender suas cerimônias, que serão detalhadas a seguir.

O **sprint** corresponde a um período fixo de tempo (normalmente duas semanas) durante o qual um incremento utilizável do produto é criado. Para viabilizar o desenvolvimento em *sprints*, as seguintes cerimônias são realizadas: **planejamento do *sprint***, **reunião diária**, **desenvolvimento do *sprint***, **revisão do *sprint*** e **retrospectiva do *sprint***.



Fonte: Adaptada de Rubin (2018).

Durante o **planejamento do sprint** (*sprint planning*), o trabalho a ser realizado é planejado. Isso envolve priorizar, entre os itens do *backlog* do produto, aqueles que devem ser desenvolvidos primeiro, analisar quais itens estão prontos para entrar no *sprint* (de acordo com a *definition of ready*) e analisar o que pode ser incluído considerando as estimativas e a capacidade da equipe. O resultado desse processo é o *backlog* do *sprint*.

O **desenvolvimento do sprint** (*sprint execution*) tem início assim que a equipe finaliza o planejamento do *sprint* e concorda com o conteúdo do próximo *sprint*. É importante ressaltar que ninguém diz à equipe de desenvolvimento qual ordem seguir ou como fazer o trabalho previsto no *backlog* do *sprint*. Os membros da equipe é que definem seu próprio trabalho e se auto-organizam para alcançar o objetivo do *sprint*.

A **reunião diária** (*daily scrum*), como o nome indica, envolve uma reunião diária de 15 minutos para que a equipe de desenvolvimento possa sincronizar as atividades e criar um plano para as próximas 24 horas. Idealmente, essas reuniões são realizadas sempre no mesmo horário. Durante elas, cada participante é solicitado a responder brevemente três perguntas:

- O que eu fiz ontem?
- O que eu vou fazer hoje?
- Tenho algum impedimento?

A **revisão do sprint** (*sprint review*) é executada no final do *sprint* para inspecionar e fazer o aceite dos itens desenvolvidos (de acordo com a *definition of done*). Como nem sempre todos os itens do *backlog* do *sprint* terão sido finalizados ou serão aceitos, a *sprint review* pode resultar em adaptações para o *backlog* do produto.

Por fim, a **retrospectiva do sprint** (*sprint retrospective*) é uma oportunidade para a equipe refletir sobre melhorias a serem aplicadas no próximo *sprint*. Essa reflexão envolve analisar o que foi bom e o que poderia melhorar e pensar em ações concretas para melhoria que poderiam ser aplicadas no próximo *sprint*.

Além das cerimônias diretamente relacionadas com o *sprint*, há ainda o **refinamento do backlog**, que envolve preparar os itens do *backlog* do produto para *sprints* futuros, refinando-os durante o *sprint* corrente para satisfazer a *definition of ready*. A equipe de desenvolvimento deveria investir de 5 a 10% do seu tempo de *sprint* para apoiar o PO em atividades de refinamento do *backlog*.



Scrum

Veja no vídeo a seguir uma breve visão geral do Scrum para recapitular o seu entendimento.

Referências

Clique aqui para acessar as referências e créditos desta aula.

